

Labreport Advanced Computer Architecture
Lab 2

Jonas Roels & Niels Van Bossche

February 25, 2024



Class: MEIsw

Contents

1 Introduction **1**

2 Reduction **1**

3 Timing comparison **1**

4 Expanding the array size **2**

1 Introduction

In this lab we will develop three versions of a program for max detection. The first approach will make use of the CPU, while both the second and third approach will use the GPU. Approach number two will use atomic operations which acquire a lock on a certain variable to guarantee mutual exclusion. The third approach will use a technique called reduction. Reduction makes it possible to reduce the amount of locking thus speeding up the program. The source code for this project can be found on `https://github.com/RoelsJonas/Advanced-Computer-Archticture-Lab-2`

2 Reduction

To implement max detection using reduction. We have each thread compare the value at double its index with the value at a certain offset from there. Initially this offset is one (thus the next value in the array). For every iteration this offset is multiplied by two, these later iterations are only carried out by threads whose index are dividable by the offset. Between each iteration to keep the results consistent all threads that are still active are synchronized using __syncThreads().

3 Timing comparison

A comparison of the processing time required to get the maximum value from an array of a certain size using each of the three approaches, can be found in Figure 1. In this graph it is easy to see that the CPU approach has a time complexity of $O(n)$ (linear). The results for the GPU approaches are less clear due to a high amount of scatter. This is due to inaccuracies in the CudaEventTimer as according to its documentation has an accuracy of approximately 0.5 µs [1].
To validate the theoretical complexity of $O(log_2(n))$ of the reduction approach it would need to be expanded to be capable of larger input sizes.
Looking at the results one would conclude that using the atomic operations approach is faster than using reduction. This seems counter intuitive as using the atomic operations only one thread can execute at a time (as they all lock on the first element of the array). This leads us to believe that the reduction approach would be faster if both approaches would be expanded to larger array sizes as here the benefits of the logarithmic complexity would be larger. As the synchronisation of all threads leads to a lower performance for small array sizes. For example an array of size 1025 and 2048 need the same amount of synchronisation even though the latter is almost double the size.



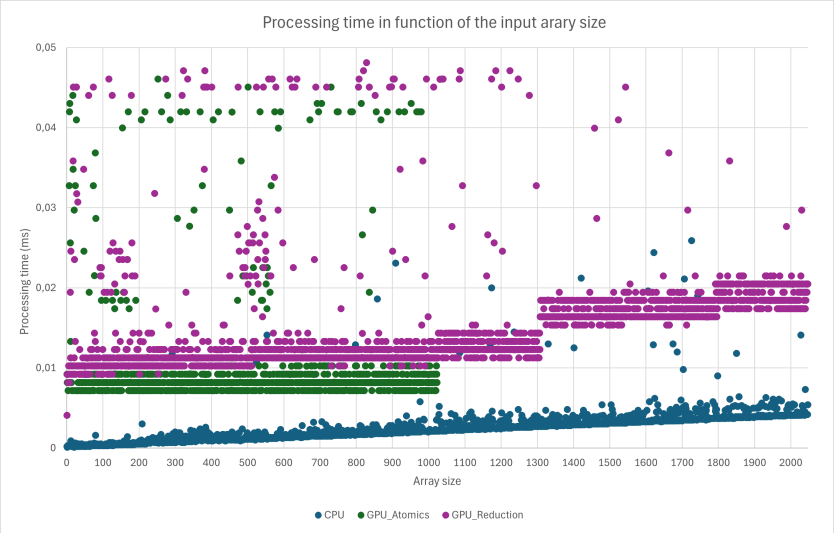


Figure 1: Comparison of the processing time of the different approaches in function of the input array size

4 Expanding the array size

A problem that arises with both GPU approaches is that it is not possible to synchronize memory access using atomic operations or __syncThreads() across multiple blocks. This combined with a maximum thread count per block of 1024 makes the max array size for the program using reduction 2048.
To increase this maximum, we could use a recursive approach. Where until we reach the required depth (and thus have a total of the size divided by 2 threads across all blocks), we have each thread create a new block with 2 threads that compare the values the thread above will compare. It is important to note that this thread has to wait until all underlying threads have completed there operations. This creates a greater overhead.

References

[1] Event management. [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/index.html